WATER LEAK LOCALIZATION SYSTEM

(LeakLens: Detecting, Classifying and Localizing Leaks in Water Distribution Networks)

A Project Report Submitted
in Partial Fulfilment of the Requirements
for the Degree of

BACHELOR OF TECHNOLOGY

by

Ashish Ranjan (Roll No. 2023BCS0027)
Nidamanuri Kedar Vaishnav (Roll No. 2023BCS0163)
Abhinandh Rajeev (Roll No. 2023BCS0074)
Kadthala Hrishith Reddy (Roll No. 2023BCS0168)

Under the Supervision of

Dr. Divya Sindhu Lekha



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
KOTTAYAM-686635, INDIA

October 2026

DECLARATION

We, **Ashish Ranjan** (**Roll No: 2023BCS0027**), **Nidamanuri Kedar Vaishnav** (**Roll No: 2023BCS0163**), **Abhinandh Rajeev** (**Roll No: 2023BCS0074**) and **Kadthala Hrishith Reddy** (**Roll No: 2023BCS0168**), hereby declare that this report entitled **"Water Leak Localization System"** submitted to Indian Institute of Information Technology Kottayam towards partial requirement of **Bachelor of Technology** in **Computer Science and Engineering** is an original work carried out by us under the supervision of **Dr. Divya Sindhu Lekha** and has not formed the basis for the award of any degree or diploma, in this or any other institution or university. We have sincerely tried to uphold the academic ethics and honesty. Whenever an external information or statement or result is used then, that have been duly acknowledged and cited.

Kottayam-686635
October 2026

Ashish Ranjan
Nidamanuri Kedar Vaishnav
Abhinandh Rajeev
Kadthala Hrishith Reddy

CERTIFICATE

This is to certify that the work contained in this project report entitled **"Water Leak Localization System"** submitted by **Ashish Ranjan** (**Roll No: 2023BCS0027**), **Nidamanuri Kedar Vaishnav** (**Roll No: 2023BCS0163**), **Abhinandh Rajeev** (**Roll No: 2023BCS0074**) and **Kadthala Hrishith Reddy** (**Roll No: 2023BCS0168**) to the Indian Institute of Information Technology Kottayam towards partial requirement of **Bachelor of Technology** in **Computer Science and Engineering** has been carried out by them under my supervision and that it has not been submitted elsewhere for the award of any degree.

Kottayam-686635
October 2026

(Dr. Divya Sindhu Lekha)
Project Supervisor

(Dr. [Name of the Head of the Department])
Head of the Department
Dept. of Computer Science & Engineering

ABSTRACT

Water distribution networks lose a large share of treated water to leaks, but a pressure drop alone does not prove a leak: a hydrant opening, a pump stopping or a tank running low lowers the pressure as well. The main aim of this project is therefore to build a software system that, from pressure, pipe-flow and customer-demand data, decides whether a network is behaving normally, names the kind of event (leak, demand change, pump or reservoir change, tank change, blocked pipe, pipe ageing) and, for a leak, ranks the junctions where it most probably is.

The work is carried out on simulated data. A scenario generator built on EPANET and WNTR produces 617 labelled 24-hour scenarios on three benchmark networks, covering leaks of five sizes at every junction and four start times, demand changes, hydrant flow, pump outages, reservoir and tank changes, blocked pipes, pipe ageing, sensor noise and leaks hidden behind other events. The method learns the sensor-noise level of each network, computes *unaccounted flow* (water that leaves a node without a customer meter recording it), summarises each scenario into fourteen features and uses a random forest to name the event. A fixed score, unaccounted flow plus a weighted pressure drop, ranks the junctions. The system is delivered through a FastAPI backend with an SQLite scenario catalogue and a Next.js dashboard with light and dark themes.

All scenarios were split once into 478 for learning and 139 for testing; every result is measured on the unseen test scenarios. The exact leak junction is ranked first for 96.7% of the test leaks (the first prototype: 58.3%), the real junction is among the first five for 98.3%, false alarms on normal days fall from 41.7% to 0.0%, and the kind of event is named correctly for 82.7% of the test scenarios. We report where the method fails: leaks inside the sensor noise, the largest leaks, and telling pump, tank, blockage and ageing events apart on a network not seen in training. All results are from simulation, not real sensors.

ACKNOWLEDGMENT

We express our sincere gratitude to our project supervisor, **Dr. Divya Sindhu Lekha**, for her guidance, encouragement and constructive criticism throughout this project. Her suggestions on how to make the evaluation honest, in particular to separate the data used for learning from the data used for testing, shaped this report.

We thank the Head of the Department and the faculty of the Department of Computer Science and Engineering, Indian Institute of Information Technology Kottayam, for the facilities and the academic environment that made this work possible. We also thank the developers of the open-source tools on which the project stands, namely EPANET, WNTR, scikit-learn, FastAPI and Next.js, and the authors of the papers cited in this report. Finally, we thank our families and friends for their constant support.

Contents

List of Figures **ix**

List of Tables **x**

1 Introduction **1**

1.1 Motivation and Background 1

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Contributions and Scope 3

1.5 Organisation of the Report 3

2 Literature Review and Related Work **4**

2.1 Reviews and Benchmarks 4

2.2 Data-Driven Methods Trained on Simulated Leaks 5

2.3 Hybrid, Graph and Topology-Based Methods 5

2.4 Classifying the Kind of Anomaly 5

2.5 Comparison and Research Gap 6

3 Hydraulic Background **7**

3.1 Water Distribution Networks as Graphs 7

3.2 EPANET and WNTR 7

3.3 Governing Equations 8

3.4 Effect of Events on the Hydraulic State 9

4 Data Generation **10**
4.1 The Networks 10
4.2 Scenario Design 10
4.3 Recorded Data and Sensor Noise 11
4.4 Ill-Conditioned Nodes and Hours 12

5 Methodology **13**
5.1 Overview 13
5.2 Residuals and Persistence 13
5.3 Learning the Noise Level 14
5.4 Unaccounted Flow 14
5.5 The Fourteen Features 16
5.6 Quiet Check 16
5.7 Cause Classification with a Random Forest 17
5.8 Localization by Ranking 17
5.9 Algorithm Summary 18

6 System Implementation **19**
6.1 Architecture 19
6.2 Technologies and Their Roles 20
6.3 Backend and Database 20
6.4 The Dashboard 21
6.5 Reproducibility 23

7 Experimental Evaluation **24**
7.1 Training and Test Protocol 24
7.2 Evaluation Metrics 25
7.3 Detection Results 25
7.4 Cause Classification Results 26
7.5 Localization Results 27
7.6 Effect of Leak Size 29
7.7 A Network Not Seen in Training 29
7.8 Training Versus Test 30

7.9 Discussion and Limitations 31

8 Conclusion and Future Work **32**

8.1 Conclusion 32

8.2 Future Work 32

References **34**

A Decision-Tree Rules **37**

B Example of an Analysis Response **39**

List of Figures

5.1 Flow of the analysis of one scenario. 13
5.2 A leak at one Net3 node. Left: unaccounted flow is large at the leak node and nearly zero elsewhere. Right: the pressure drop is also high at its neighbours, so pressure alone is less precise. 15
5.3 Pressure at one Net3 node on a normal day (blue) and in three scenarios (red): a leak, a demand surge and a pump outage. 15

6.1 End-to-end architecture, from network models to the user's screen. 19
6.2 The Investigate screen (light theme) after analysing a leak: choose a scenario on the left, the network map in the centre, the verdict and reasons on the right. 22
6.3 The same screen in the dark theme, with the predicted node and its search region. 23

7.1 The confusion matrix as a picture. 27
7.2 Top-1, Top-3 and Top-5 of each localization method on the test leaks. 28
7.3 Detection and exact-node success by leak size. 29

List of Tables

2.1 Summary of related work and positioning of this project. 6

3.1 What each kind of event does to the network. 9

4.1 The three benchmark networks. 10
4.2 The scenario kinds and what is changed. 11

5.1 The fourteen features of a scenario. 16

6.1 Technologies used. 20
6.2 Backend endpoints. 21

7.1 Training and test scenarios by kind of event. 24
7.2 Detection on the 139 test scenarios (12 normal days). 25
7.3 Per-kind precision, recall and F_1 on the test scenarios. 26
7.4 Confusion matrix on the test scenarios (rows: truth; columns: system; a dot is 0). 26
7.5 Leak localization on the 60 scored test leaks. 27
7.6 Top-1 and Top-5 by leak type, node type and network (final method). 28
7.7 Effect of hole area on the test leaks. 29
7.8 Leave-one-network-out results. 30
7.9 Evolution of the method. 30

Chapter 1

Introduction

Water distribution networks (WDNs) carry treated water from sources to customers through pipes, pumps, tanks and valves. Part of that water never reaches a customer because pipes leak. This chapter explains why leak localization matters, states the problem, lists the objectives and contributions of the project, and describes how the rest of the report is organised.

1.1 Motivation and Background

Water utilities lose a large share of the water they produce through leaks and other unmetered use, usually called non-revenue water. The literature reports leakage from about 3% to over 50% depending on maintenance [1], and our project presentation quotes about 40% for developing countries. Leaks waste water and energy, damage roads and buildings and can let contamination enter the pipes.

Finding leaks by inspecting streets or listening to pipes with acoustic equipment is slow and labour-intensive [3]. Pressure and flow sensors scale better to large networks, but a pressure drop alone is ambiguous. A hot day, a hydrant, a stopped pump or a tank running low also lowers pressure, so a fixed pressure alarm either misses leaks or raises many false alarms. Software methods that combine a hydraulic model with data analysis are therefore considered a promising direction [1, 2].

1.2 Problem Statement

Given pressure at nodes, flow in pipes and, where available, customer demand at nodes, for a water network whose layout is known, the system must (i) decide whether the network is behaving abnormally, (ii) decide what kind of event it is, and (iii) if it is a leak, rank the candidate nodes and report the most probable location with supporting evidence.

Definition 1.2.1 (Leak localization task)**.** Let $\mathcal{V}$ be the set of junctions of a network and $\mathcal{D}$ the observed pressure, flow and demand time series of one day. A *localizer* is a function that maps $\mathcal{D}$ to an ordered list of the junctions in $\mathcal{V}$. It is *correct at rank* k if the true leak junction is among the first k entries.

1.3 Objectives

The objectives set in the project proposal for Semester 1 were to select one or two representative networks, generate normal and single-leak scenarios varying leak location, size and start time, build datasets with ground truth, establish an interpretable localization baseline, and provide a web prototype. Because the relevance of a leak detector depends on telling a leak from other hydraulic changes, the work was extended with the following objectives:

1. generate scenarios on three networks, including demand, pump, reservoir, tank, blockage, ageing and sensor-noise scenarios;
2. replace the fixed pressure alarm by a detector that learns the noise level of each network;
3. classify the cause of an event, not only detect it;
4. localize leaks with a method whose error is measured as distance on the pipe graph;
5. evaluate honestly on scenarios that were not used for learning;
6. deliver the result as an easy-to-use web dashboard.

1.4 Contributions and Scope

The contribution of this project is the *integration* of known ingredients and a controlled, reproducible study; it does not claim a new algorithm. In particular:

- a reproducible scenario generator with ground truth on three benchmark networks;
- a noise-aware detector with a two-hour persistence rule and a quiet check;
- a cause classifier built on fourteen physics-based features;
- a localizer based on *unaccounted flow*, whose error is measured in pipe hops and metres;
- an 80/20 hold-out evaluation in which near-copies of a scenario stay on the same side of the split;
- a FastAPI backend with an SQLite catalogue and a Next.js dashboard with light and dark themes.

The scope is limited to simulated data, single leaks, and demand measured at every node. Missing or sparse sensors, multiple simultaneous leaks and real utility data are left for Semester 2.

1.5 Organisation of the Report

Chapter 2 reviews related work. Chapter 3 gives the hydraulic background and the governing equations. Chapter 4 describes the networks, scenarios and data. Chapter 5 presents the method. Chapter 6 describes the implementation of the backend and dashboard. Chapter 7 gives the evaluation protocol and results. Chapter 8 concludes and outlines future work.

Chapter 2

Literature Review and Related Work

This chapter surveys recent work on leak detection and localization in water distribution networks and positions the project against it. Reviews and benchmarks are covered first, then data-driven methods trained on simulated data, graph and topology-based methods, and work on classifying anomalies.

2.1 Reviews and Benchmarks

Sousa et al. [1] review scientific, industrial and commercial developments in leakage management, compare hardware-based, software-based and hybrid techniques, and report trade-offs in scalability, data dependency, uncertainty and localization accuracy. Romero-Ben et al. [2] review the move from model-based to data-based leak localization and compare promising methods on a benchmark. Vaquet et al. [3] survey machine learning in water distribution networks, including a structured review of leakage detection and localization and evaluation benchmarks.

Benchmarks make comparison possible. LeakDB [4] provides simulated but realistic leakage scenarios built with WNTR. The Battle of the Leakage Detection and Isolation Methods (BattLeDIM) compared methods on a virtual network with SCADA measurements and concluded that further development of methods and additional open benchmark problems are needed [5].

2.2 Data-Driven Methods Trained on Simulated Leaks

Because real leaks seldom come with ground truth, many methods are trained on hydraulic simulations. Lučin et al. [6] train a random forest on simulated leaks with random location, size and demand uncertainty from pressure sensors, and add pipe segmentation so that the leak is located along a pipe and not only at a node. Marvin et al. [7] generate pressure data with randomly varied leak location, area and start time, add pressure noise, and compare a random forest with histogram-based gradient boosting, which performs better. Basnet et al. [8] study supervised multilayer-perceptron and convolutional models for localization under different leak complexities on a benchmark network. Rajabi et al. [9] turn pressure data into images by kriging interpolation and use a conditional generative adversarial network for detection and localization.

These studies motivate our use of simulated, labelled scenarios and a tree-based classifier, and they show that random forests and gradient boosting are common choices for this tabular problem.

2.3 Hybrid, Graph and Topology-Based Methods

Soldevila et al. [10] combine expert knowledge with data-driven models: leaks are detected on the inlet flow and localized by a classifier that groups locations the sensors cannot tell apart; it is evaluated on one network. Irofti et al. [11] combine graph-based interpolation with dictionary-learning classification on the L-Town benchmark. Leonzio et al. [12] select monitoring nodes with graph centrality and show that the sensor strategy that works best depends on topology. Zhang and Fink [13] pre-train a graph neural network to imitate a max-flow algorithm so that it generalizes better, and van Gemert et al. [14] study how sensor placement affects graph-neural-network leak detection.

2.4 Classifying the Kind of Anomaly

Most localization studies assume that a leak is the only possible event. Melnikov et al. [15] are closer to our question: they test whether a network behaves normally

with a statistical method, classify anomalies as abrupt leaks, incipient leaks or sensor malfunctions, and give a coarse pre-localization. Our work extends the set of alternatives to demand changes, pump and reservoir changes, tank changes, blocked pipes and pipe ageing.

2.5 Comparison and Research Gap

Table 2.1: Summary of related work and positioning of this project.

Work	Approach and data	Main limitation
Lučin et al. 2021 [6]	Random forest on simulated leaks, pipe segmentation	Leak is the only event considered
Marvin et al. 2023 [7]	Gradient boosting vs. random forest, WNTR data	Single network; leak only
Basnet et al. 2023 [8]	MLP and CNN on L-Town	Leak only; black-box models
Soldevila et al. 2022 [10]	Inlet-flow test and clustered classifier	One network; no graph reasoning
Leonzio et al. 2026 [12]	Centrality-guided sensor selection, autoencoder	Detection, not localization
Zhang and Fink 2026 [13]	Algorithm-informed graph neural network	Learned; hard to interpret
Melnikov et al. 2025 [15]	Multivariate statistics; anomaly classes	Coarse localization; preprint
This project	Noise-aware detection, unaccounted flow, random forest, ranking; three networks	Simulation only; demand meters assumed

The studies above typically evaluate one network at a time, treat detection and localization separately, or rely on models that are hard to explain. We did not find a work that combines noise-aware detection, flow-balance reasoning, classification of the kind of event, a graph-distance error measure and an interpretable ranked output with an evaluation on held-out scenarios across networks. Our search was not exhaustive.

Chapter 3

Hydraulic Background

This chapter introduces the hydraulic concepts and equations used to generate the data and to interpret it.

3.1 Water Distribution Networks as Graphs

A network is a graph whose vertices are *junctions* (where customers draw water), *reservoirs* (sources with fixed head) and *tanks* (storage), and whose edges are *pipes*, *pumps* and *valves*. Every edge has a start and an end node, and the flow in it is signed: positive from start to end.

Definition 3.1.1 (Pressure, flow and demand)**.** The *pressure* $P_i(t)$ at node i and hour t is the pressure head in metres. The *flow* $Q_p(t)$ is the volume of water passing pipe p per second (m^3/s). The *demand* $D_i(t)$ is the water delivered to the customers of node i (m^3/s).

3.2 EPANET and WNTR

EPANET [16] is the standard hydraulic simulator for pressurised pipe networks and defines the `.inp` network format. WNTR [17] is a Python package that loads such files and adds leaks, controls, demand patterns and pump outages, and it provides its own simulator. We use WNTR's simulator with *pressure-dependent*

demand (PDD): when pressure is low, the delivered demand falls below the requested demand. This is more realistic than demand-driven analysis for leaks and pump outages.

3.3 Governing Equations

Definition 3.3.1 (Continuity at a junction)**.** For a junction i, let $\text{sign}(p, i) = +1$ if pipe p ends at i and -1 if it starts there. The *net inflow* is

$$\text{div}_i(t) = \sum_{p} \text{sign}(p, i)\, Q_p(t). \tag{3.1}$$

Conservation of mass requires

$$\text{div}_i(t) = D_i(t) + Q_{\ell,i}(t), \tag{3.2}$$

where $Q_{\ell,i}$ is the leak flow at node i (0 if there is no leak).

A leak is modelled as an orifice. With discharge coefficient $C_d = 0.75$, hole area A, gravitational acceleration $g = 9.81$ m/s^2 and pressure head P_i,

$$Q_{\ell,i}(t) = C_d\, A\, \sqrt{2\, g\, P_i(t)} \qquad (t \geq t_{\text{start}}), \tag{3.3}$$

so the leak flow grows with the pressure. Customer demand is the base demand scaled by a daily pattern and a scenario factor,

$$D_i(t) = D_{\text{base},i}\; m(t)\; f. \tag{3.4}$$

The pressure lost in a pipe of length L, diameter d and roughness coefficient C carrying flow Q follows the Hazen–Williams law,

$$h_f = \frac{10.67\, L\, Q^{1.852}}{C^{1.852}\, d^{4.87}}, \tag{3.5}$$

so a smaller C (an older, rougher pipe) loses more pressure for the same flow.

3.4 Effect of Events on the Hydraulic State

Table 3.1: What each kind of event does to the network.

Event	Pressure	Flow balance	Water lost?
Leak	Largest drop at one node, decaying with distance	Unmetered loss at that node	Yes
Demand change	Drop near heavy use	Extra flow is metered	No
Pump / reservoir	Wide drop or shift over the served area	Pump flow falls; tanks supply	No
Tank change	Shift near the tank	Tank flow changes sign or size	No
Blocked pipe	Down downstream, may rise upstream	Flow reroutes	No
Pipe ageing	Slow, wide loss, largest where flow is high	No node imbalance	No

A leak is therefore *local* and *unmetered*; the other events are either metered or spread over many nodes. This difference is the basis of the method of Chapter 5.

Chapter 4

Data Generation

4.1 The Networks

Three standard EPANET benchmark networks of different size and structure are used, so that generalization can be tested.

Table 4.1: The three benchmark networks.

Network	Junctions	Analysed	Pumps	Tanks	Reservoirs	Scenarios	τ_P (m)
NET1	9	9	1	1	1	155	0.498
NET2	35	35	0	1	0	171	0.535
NET3	92	90	2	3	2	291	0.582

Net2 has no reservoir and no pump; one junction with negative demand acts as its supply. Net3 nodes 61 and 601 and the hour 21:00 are ignored (Section 4.4).

4.2 Scenario Design

Definition 4.2.1 (Scenario)**.** A *scenario* is one 24-hour simulation of a network in which one thing (or a few things together) differs from the normal day. The *baseline* is the normal day with nothing changed. Each scenario is stored with its *true answer*, namely the kind of event and, for leaks, the leak node. The true answer is used only to label training data and to score results.

Table 4.2 lists the 617 scenarios.

Table 4.2: The scenario kinds and what is changed.

Kind	Count	What is changed
Leak	299	Hole at one junction; areas 0.0001, 0.0003, 0.001, 0.003, 0.01 m^2 (tiny to burst); every junction; start at 2, 8, 12 or 20 h; sensor noise; hidden behind a global demand surge, a hydrant, a closed pipe or a pump outage
Demand change	60	One node $\times 1.5$ or $\times 0.5$; all nodes $\times 1.3$ or $\times 0.7$; a hydrant of 0.01 m^3/s for four hours
Pump / reservoir	54	A pump off for 2, 4 or 6 h while it normally runs; reservoir head $\pm 2, \pm 5, \pm 10$ m
Tank change	60	A tank starts 10–90% full, or its pipe is closed at 10:00
Blocked pipe	48	A busy pipe closed from 10:00, or throttled
Pipe ageing	42	Roughness coefficient C multiplied by 0.6–0.95
Normal day	54	Nothing changes; sensor noise only (or none)

Most non-leak scenarios also have a noisy twin. The scenarios for each network are generated from the network's own layout, so every node, pipe, pump, tank and reservoir identifier is valid for that network; blockage candidates are tested for hydraulic convergence before they are included.

4.3 Recorded Data and Sensor Noise

For every scenario three tables are saved with one row per hour (hours 0 to 24): pressure at every node, flow in every pipe, and the demand delivered at every node (what a customer meter would read; the leak flow is not included). In noisy scenarios Gaussian errors with fixed random seeds are added to the saved readings: low noise is 0.05 m on pressure and 0.5% on flows and demands; high noise is 0.2 m and 2%. The baseline is kept clean because it stands for the expectation of the hydraulic model.

4.4 Ill-Conditioned Nodes and Hours

A tiny 1% demand probe is simulated on each network. A node whose pressure swings by more than 5 m, and an hour at which 90% of the nodes swing by more than 0.5 m, are *ill-conditioned*: they are artefacts of pump switching and not reliable signals. They are ignored in all analysis. In Net3 these are the nodes 61 and 601 (the outlet of a pump that is off) and the hour 21:00. Leaks at these nodes are therefore not scored. Scenarios whose hydraulics did not converge were discarded (three Net3 blockage candidates).

Chapter 5

Methodology

5.1 Overview

The method has two stages. Stage 1 decides whether anything is above the sensor noise. Stage 2 decides what kind of event it is and, for a leak, where. Figure 5.1 shows the flow.

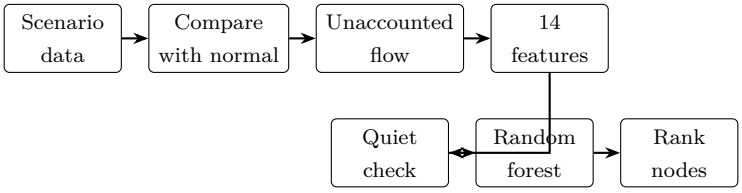


Figure 5.1: Flow of the analysis of one scenario.

5.2 Residuals and Persistence

Let $P_i^0(t)$ and $P_i^1(t)$ be the pressure of node i at hour t on the normal day and in the scenario. The *residual* is $r_i(t) = P_i^0(t) - P_i^1(t)$; a positive residual means the pressure is lower than normal. To ignore one-off errors a change counts only if it lasts two consecutive hours.

Definition 5.2.1 (Persistent peak)**.** For a series $x(t)$ the *persistent peak* is

$$\text{peak}(x) = \max_t \ \min\{x(t),\, x(t{+}1)\}. \tag{5.1}$$

Remark 5.2.2. For the series 0.1, 0.2, 6.5, 6.7, 6.6, 6.4, 0.3 the pairwise minima are $0.1, 0.2, 6.5, 6.6, 6.4, 0.3$ and the persistent peak is 6.6. A single one-hour spike of 9.0 between 0.2 and 0.3 has a persistent peak of only 0.3.

5.3 Learning the Noise Level

Each network has normal runs that contain only sensor noise. The noise levels are learned only from the *training* normal runs (14 per network). The pressure threshold is

$$\tau_P = \max\{0.1 \text{ m},\ 4\,\sigma_P\}, \tag{5.2}$$

where σ_P is the standard deviation of the pressure residuals over the analysed nodes and hours. The noise levels of each pipe flow ($\sigma_{f,p}$), each node demand ($\sigma_{D,i}$) and each node's unaccounted flow ($\sigma_{a,i}$) are learned in the same way. The learned values are τ_P =0.50, 0.53 and 0.58 m for Net1, Net2 and Net3.

Definition 5.3.1 (Noise unit)**.** A *noise unit* expresses a measurement as a multiple of its learned noise limit, so 1 means "exactly at the noise limit". The pressure drop of node i and its rise are

$$d_i = \frac{\max\{0, \text{peak}(r_i)\}}{\tau_P}, \qquad g_i = \frac{\max\{0, \text{peak}(-r_i)\}}{\tau_P}. \tag{5.3}$$

5.4 Unaccounted Flow

Definition 5.4.1 (Unaccounted flow)**.** Let Δ denote the scenario value minus the normal-day value. The *unaccounted flow* of node i is the change in net inflow minus the change in metered demand,

$$a_i(t) = \Delta\,\text{div}_i(t) - \Delta D_i(t). \tag{5.4}$$

In noise units it is $a_i = \max\{0, \text{peak}(a_i(t))\}/(4\,\sigma_{a,i})$.

Proposition 5.4.2. *In noise-free data, $a_i(t) = Q_{\ell,i}(t)$: the unaccounted flow of a node equals its leak flow, and it is* 0 *at every node without a leak.*

Proof. By continuity (3.2), $\mathrm{div}_i = D_i + Q_{\ell,i}$ in the scenario and $\mathrm{div}_i = D_i$ on the normal day (no leak). Subtracting, $\Delta\mathrm{div}_i = \Delta D_i + Q_{\ell,i}$. Substituting into (5.4) gives $a_i = Q_{\ell,i}$. If node i has no leak then $Q_{\ell,i} = 0$ and so $a_i = 0$. □

Corollary 5.4.3. *A change in customer demand alone gives $a_i = 0$, because the metered change ΔD_i cancels the extra net inflow. A demand change is therefore never mistaken for a leak when demands are metered.*

Remark 5.4.4. Without nodal demand meters the second term of (5.4) is unavailable; a demand change then appears as unaccounted flow and may be mistaken for a small leak. We measure this effect in Chapter 7.

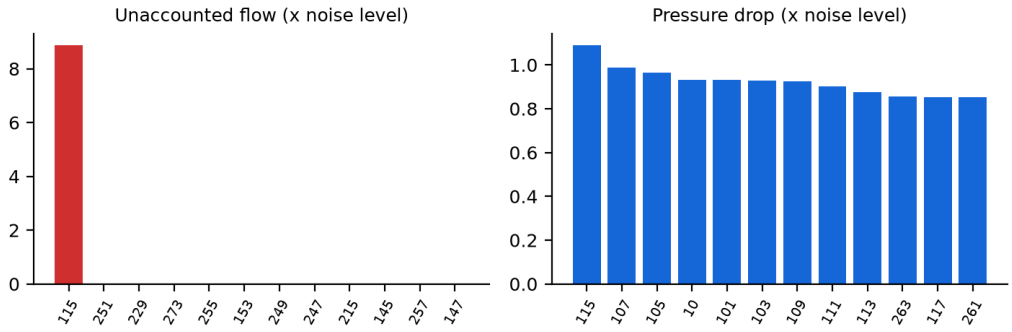


Figure 5.2: A leak at one Net3 node. Left: unaccounted flow is large at the leak node and nearly zero elsewhere. Right: the pressure drop is also high at its neighbours, so pressure alone is less precise.

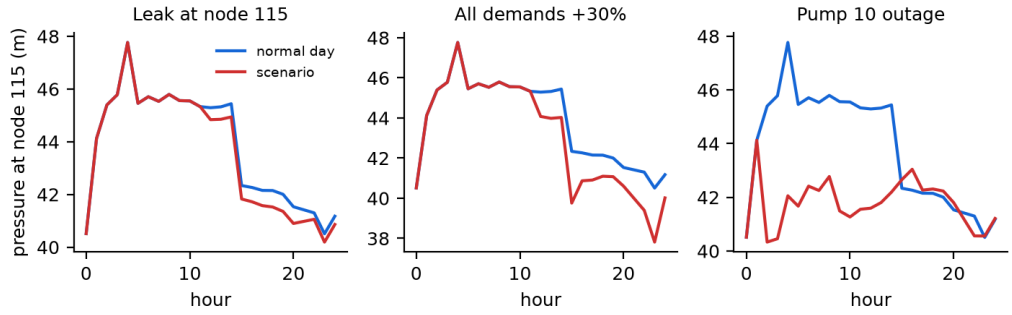


Figure 5.3: Pressure at one Net3 node on a normal day (blue) and in three scenarios (red): a leak, a demand surge and a pump outage.

5.5 The Fourteen Features

Each scenario is summarised by fourteen numbers computed from the node and pipe signals of the *whole network*. Here $z_p = \text{peak}\,|Q_p^1 - Q_p^0|/(4\sigma_{f,p})$ is the flow change of pipe p in noise units, z_i^D the same for node demands, $\Delta S(t)$ the change in total reservoir supply, $\Delta T(t)$ the change in net tank flow, D_{tot} the mean total customer demand and k runs over pumps.

Table 5.1: The fourteen features of a scenario.

#	Name	Meaning	Formula
1	`drop_ratio`	Largest pressure drop at any junction	$\max_i d_i$
2	`frac_hit`	Share of junctions that lost pressure	$\text{mean}(d_i > 1)$
3	`median_ratio`	Typical (median) junction drop	$\text{median}_i\, d_i$
4	`conc_drop`	Is the drop concentrated at one node?	$\max d_i / \sum d_i$
5	`rise_ratio`	Largest pressure rise at any junction	$\max_i g_i$
6	`frac_rise`	Share of junctions that gained pressure	$\text{mean}(g_i > 1)$
7	`flow_frac`	Share of pipes with changed flow	$\text{mean}(z_p > 1)$
8	`source_change`	Change in reservoir supply over demand	$\max_t \lvert\Delta S(t)\rvert / D_{tot}$
9	`tank_change`	Change in tank net flow over demand	$\max_t \lvert\Delta T(t)\rvert / D_{tot}$
10	`pump_drop`	Pump flow lost (1 = pump off)	$\max(Q_k^0 - Q_k^1) / \max Q_k^0$
11	`unacc_ratio`	Largest unaccounted flow (noise units)	$\max_i a_i$
12	`unacc_conc`	Unaccounted flow concentrated at one node?	$\max a_i / \sum a_i$
13	`demand_frac`	Share of nodes with changed metered demand	$\text{mean}(z_i^D > 1)$
14	`demand_total`	Total demand change over demand (signed)	$\sum_i \Delta D_i / D_{tot}$

All features are scaled by noise limits or by total demand, so they are comparable between networks.

5.6 Quiet Check

Definition 5.6.1 (Quiet scenario)**.** A scenario is *quiet* if every signal is within the noise: $\max_i d_i < 1$, $\max_i g_i < 1$, $\max_i a_i < 1$, flow_frac < 0.1 and demand_frac $= 0$.

A quiet scenario is labelled *normal day* without calling the classifier. This removes false alarms caused by sensor noise; its consequence is that leaks smaller than the noise cannot be detected.

5.7 Cause Classification with a Random Forest

A *random forest* [6, 7] of 300 decision trees maps the fourteen features to a probability for each of the seven kinds. Each tree is trained on its own bootstrap sample of the training scenarios, drawn with replacement, and at each question only a random subset of $\lfloor\sqrt{14}\rfloor = 3$ features may be used. With `class_weight="balanced"` (leaks are about half of the training scenarios) each tree's sample contains about equal shares of the seven kinds. Trees are grown until the groups are pure (no depth limit), the impurity is measured by the Gini index $1 - \sum_k \pi_k^2$, and the random seed is fixed so that training is repeatable. The probability of a kind is the average over the trees of the share of that kind in the end-point reached; because the trees are grown until pure, this is effectively the vote share. The verdict is the kind with the highest probability, and the *confidence* is that probability (High if at least 85%, Medium if at least 60%, otherwise Low). A depth-4 decision tree is trained alongside; its rules are readable (Appendix A) but it is not used for the verdict. No hyper-parameter was tuned.

5.8 Localization by Ranking

If the verdict is leak, every analysed junction receives a score. With $A_i = a_i / \max_j a_j$ and $D_i = d_i / \max_j d_j$ (and $A_i = 0$ for all nodes if $\max_j a_j < 1$),

$$\text{score}_i = A_i + 0.3\, D_i. \tag{5.5}$$

The junctions are sorted by score; the first is the *Top-1* prediction and the next ones are where to look if it is wrong. Unaccounted flow is the main signal because it points to one node; the pressure drop only breaks ties, because it also spreads to neighbouring nodes. The ranking is a fixed formula and is not trained. Smoothing the score over neighbouring nodes was tested and gave no improvement, so it is not used.

Definition 5.8.1 (Localization error and search region)**.** The *error* of a prediction $\hat{n}$ for a true node n is the number of pipes on the shortest path between them (*hops*) and the length of that path in metres. The *search region* is $\hat{n}$ together with the nodes directly connected to it.

5.9 Algorithm Summary

Step 1. Read the pressure, flow and demand tables of the scenario and of the baseline; drop the ignored hour.

Step 2. Compute residuals, persistent peaks and noise-unit drops d_i and rises g_i.

Step 3. Compute net inflows from pipe flows (one matrix product) and the unaccounted flow a_i.

Step 4. Summarise into the fourteen features.

Step 5. If the scenario is quiet, answer “normal day”.

Step 6. Otherwise the random forest gives the probability of each kind; the highest is the verdict.

Step 7. If the verdict is leak, compute (5.5), sort, and report the top five nodes and the search region.

Chapter 6

System Implementation

6.1 Architecture

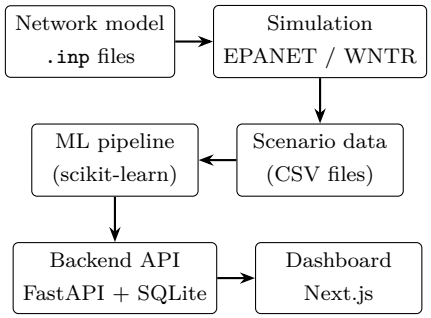


Figure 6.1: End-to-end architecture, from network models to the user's screen.

The code is organised as a monorepo: `simulation/` builds scenarios and runs the simulator, `ml_pipeline/` holds data loading, features, detector, classifier, localizer and evaluation, `backend/` exposes the results, `frontend/` is the dashboard and `data/` holds networks, simulated scenarios and result files.

6.2 Technologies and Their Roles

Table 6.1: Technologies used.

Technology	Role
EPANET 2.2, WNTR 1.5	Network files; simulation of every scenario with pressure-dependent demand
Python, pandas, NumPy	Data tables and numerical computation
NetworkX	Pipe graph, shortest paths (hops, metres) and neighbours (search region)
scikit-learn 1.9, joblib	Random forest and decision tree; evaluation metrics; saving the model
FastAPI, Uvicorn, Pydantic	REST endpoints and validated response schemas
SQLAlchemy, SQLite	Scenario catalogue (kind, node, size, train/test tag)
Next.js 16, React 19, TypeScript	The dashboard
Tailwind CSS 4	Styling with colour tokens for light and dark themes
React Flow, Recharts, lucide-react, axios	Network map, charts, icons and API calls

6.3 Backend and Database

The backend creates an SQLite table `scenarios` at start-up from the generated files, so the dashboard always lists exactly what exists. Its columns are the scenario identifier, network, name, group, kind, true leak nodes, a JSON field of parameters (kind, node, size, noise, hidden-by, train/test tag) and the data folder. The time series stay in CSV files. Scenario identifiers are validated against a whitelist of characters before any file is read.

Table 6.2: Backend endpoints.

Endpoint	Returns
`GET /api/networks`	Networks with junction and scenario counts
`GET /api/scenarios?network=`	Scenario list with parameters and train/test tag
`GET /api/network?network=`	Node positions and pipes for the map
`GET /api/analyze/{id}`	Verdict, probabilities, features, ranked nodes, search region, affected nodes, per-node and per-pipe numbers, true answer
`GET /api/evaluation`	The saved test results

When the dashboard requests an analysis, the backend loads the saved tables of the scenario and the baseline, runs the analysis of Section 5.9 (about 22 ms), and returns one JSON object. The simulation is *not* run at that time: it was run beforehand and its results were saved.

6.4 The Dashboard

The first prototype exposed two long drop-down lists and technical vocabulary. The redesigned dashboard, named LeakLens, follows these principles.

- **Guided choice:** three steps (network, what happened, narrow it down) with a counter of matching scenarios, node search and *Pick on map*, size and condition filters, and next/previous and random-scenario buttons.
- **Plain-language verdict:** a headline, a confidence label, a "what to do" sentence and reasons written from the measurements. The reasons are fixed templates that appear when a feature passes a limit (for example, a pump-flow loss above 0.5 or an unaccounted-flow ratio of at least 1).
- **Honest testing built in:** every scenario carries an *Unseen test* or *Training* tag, unseen scenarios are shown by default, and analysing a training scenario shows a warning that a correct answer proves little.
- **Explained terms, themes and accessibility:** dotted-underline words show definitions, light and dark themes follow the system setting, controls

are labelled and keyboard accessible, and the layout adapts to phones.

- **Performance and Guide tabs:** the test results of Chapter 7 in plain language, and a glossary.

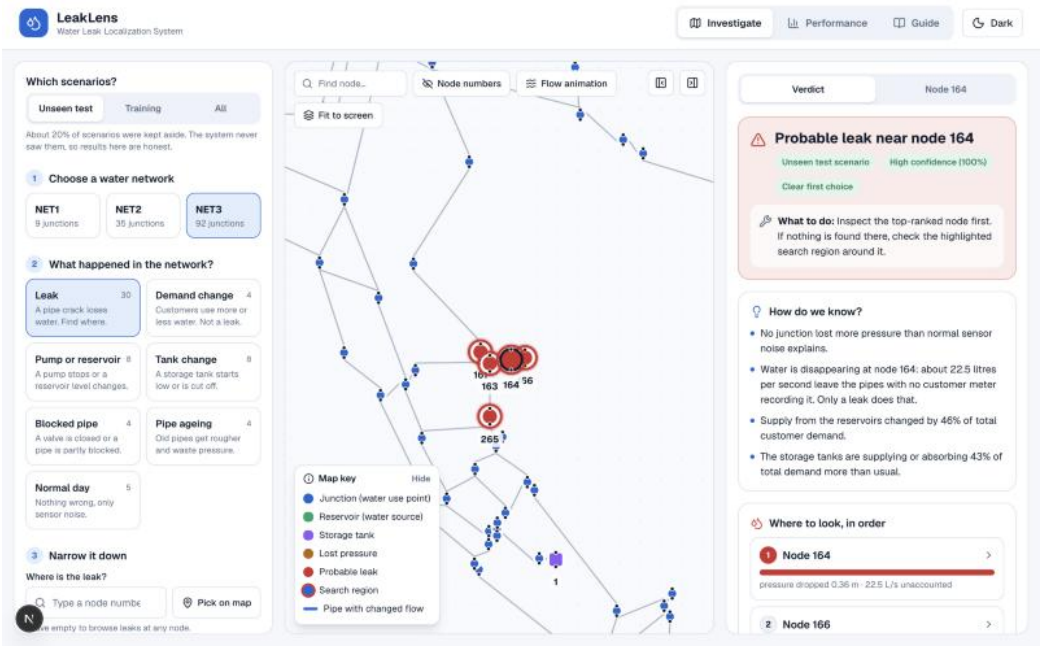


Figure 6.2: The Investigate screen (light theme) after analysing a leak: choose a scenario on the left, the network map in the centre, the verdict and reasons on the right.

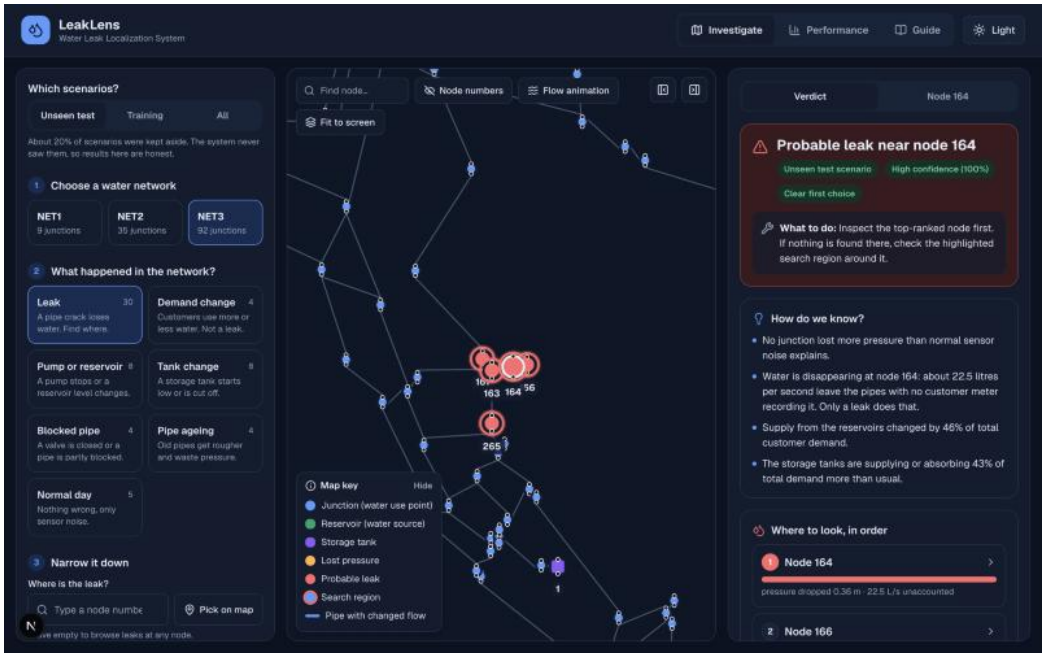


Figure 6.3: The same screen in the dark theme, with the predicted node and its search region.

6.5 Reproducibility

The scenarios, the split and the model are generated by scripts with fixed random seeds, in the order: `build_scenarios.py`, `generate_dataset.py`, `train.py`, `evaluate.py`; the backend and the dashboard are then started with `uvicorn main:app` and `npm run dev`. Re-training on the same rows reproduces the same forest.

Chapter 7

Experimental Evaluation

7.1 Training and Test Protocol

Definition 7.1.1 (Hold-out split)**.** All scenarios are split once (random seed 42) into a *training set* (about 80%) used for learning, and a *test set* (about 20%) used only for measuring performance. Scenarios that are near-copies stay on the same side: all leaks at one junction (every size, start time, noise level and "hidden-behind" variant), and a scenario with its sensor-noise twin. The split is stratified by network and kind of event.

Table 7.1: Training and test scenarios by kind of event.

Kind of event	All	Train	Test
Normal day	54	42	12
Leak	299	232	67
Demand change	60	48	12
Pump / reservoir	54	42	12
Tank change	60	48	12
Blocked pipe	48	36	12
Pipe ageing	42	30	12
Total	617	478	139

The noise thresholds and the random forest are fitted on the 478 training sce-

narios only, and every result below is measured on the 139 test scenarios. Because groups of near-copies are kept together, the share is 77.5% and 22.5%, not exactly 80% and 20%. The ranking weight (0.3) and the quiet-check cut-offs were chosen during development while looking at all scenarios, before the split existed, and were not re-tuned.

7.2 Evaluation Metrics

Detection. A scenario is positive if any abnormal event happened. With true positives (TP), false positives (FP), false negatives (FN) and true negatives (TN),

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}, \tag{7.1}$$

$$F_1 = \frac{2\,\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}, \qquad \text{false-alarm rate} = \frac{FP}{FP + TN}. \tag{7.2}$$

Classification. Accuracy is the share of correct answers; per-kind precision, recall and F_1 are computed with TP the diagonal of the confusion matrix, FP the column total minus TP and FN the row total minus TP; macro-F_1 is the plain average of the seven F_1 values.

Localization. For a test leak with true node n_s let r_s be its position in the sorted list. Then

$$\text{Top-}k = \frac{1}{N}\sum_{s=1}^{N} \mathbf{1}[r_s \le k], \qquad \text{mean hops} = \frac{1}{N}\sum_{s=1}^{N} \text{hops}(\hat{n}_s, n_s). \tag{7.3}$$

End-to-end success counts a leak only if the verdict is leak and $r_s = 1$.

7.3 Detection Results

Table 7.2: Detection on the 139 test scenarios (12 normal days).

Detector	Precision	Recall	F_1	False alarms	TP	FP	FN	TN
Fixed 0.5 m alarm	95.7%	87.4%	91.4%	41.7%	111	5	16	7
Learned pressure threshold	100.0%	77.2%	87.1%	0.0%	98	0	29	12
Full system	100.0%	93.7%	96.7%	0.0%	119	0	8	12

The full system has precision 100.0%, recall 93.7% and F_1 96.7% with a false-alarm rate of 0.0%, against 41.7% for the fixed 0.5 m alarm. With only 12 normal test days the true false-alarm rate is not exactly zero.

7.4 Cause Classification Results

Accuracy on the test scenarios is **82.7%** (115 of 139); on the training scenarios it is 95.6%. The macro-F_1 is 76.4%.

Table 7.3: Per-kind precision, recall and F_1 on the test scenarios.

Kind	TP	FP	FN	Precision	Recall	F_1
Normal day	12	8	0	60.0%	100.0%	75.0%
Leak	60	1	7	98.4%	89.6%	93.8%
Demand change	12	0	0	100.0%	100.0%	100.0%
Pump / reservoir	8	4	4	66.7%	66.7%	66.7%
Tank change	8	6	4	57.1%	66.7%	61.5%
Blocked pipe	9	0	3	100.0%	75.0%	85.7%
Pipe ageing	6	5	6	54.5%	50.0%	52.2%

Table 7.4: Confusion matrix on the test scenarios (rows: truth; columns: system; a dot is 0).

Truth \ system	Normal	Leak	Demand	Pump	Tank	Blocked	Ageing	Total
Normal day	12	.	.	.	.	.	.	12
Leak	6	60	.	.	1	.	.	67
Demand change	.	.	12	.	.	.	.	12
Pump / reservoir	2	.	.	8	2	.	.	12
Tank change	.	.	.	.	8	.	4	12
Blocked pipe	.	1	.	.	1	9	1	12
Pipe ageing	.	.	.	4	2	.	6	12

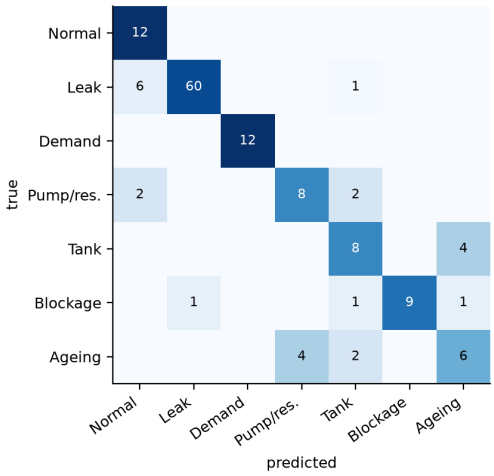


Figure 7.1: The confusion matrix as a picture.

Leaks and demand changes are recognised reliably. Pump, tank, blockage and ageing events are sometimes mistaken for each other, because each lowers pressure over a wide area without losing water. The leak recall is 89.6%; all seven test leaks that were not named leaks are at Net3 node 601, an ignored pump-outlet node, and only 1.4% of non-leak events were wrongly called a leak.

7.5 Localization Results

Table 7.5: Leak localization on the 60 scored test leaks.

Method	Top-1	Top-3	Top-5	Mean hops	≤ 1 hop	Metres
First prototype	58.3%	70.0%	71.7%	3.35	58.3%	1774
Final method	96.7%	98.3%	98.3%	0.13	96.7%	60
Final + smoothing	96.7%	98.3%	98.3%	0.13	96.7%	60
Final, no demand meters	93.3%	98.3%	98.3%	0.72	93.3%	303
Pressure only	60.0%	70.0%	70.0%	3.05	61.7%	1208
Pressure only + smoothing	50.0%	63.3%	65.0%	3.18	58.3%	1244

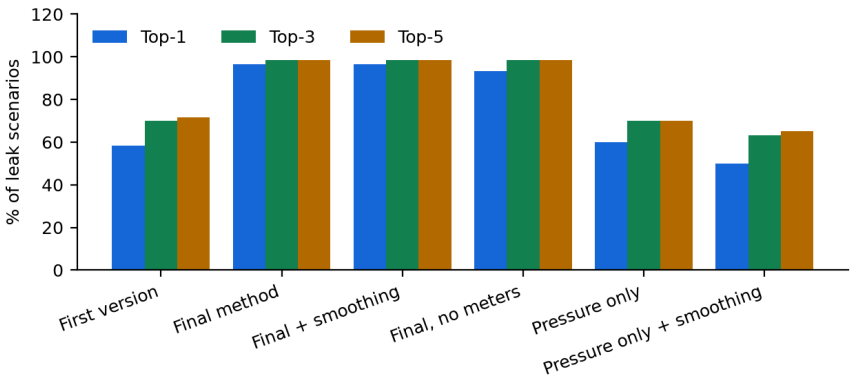


Figure 7.2: Top-1, Top-3 and Top-5 of each localization method on the test leaks.

The exact node is ranked first for 58 of 60 test leaks (96.7%), against 35 of 60 (58.3%) for the first prototype. The final method is on average 0.13 pipes (about 60 m) from the true node and 96.7% of the leaks are found at the node or its direct neighbour; end-to-end success is 96.7%. Without demand meters the Top-1 is 93.3%, with pressure alone it is 60.0%, and neighbour smoothing gives 96.7%. With only 60 test leaks, one miss changes a percentage by about 1.7 points.

Table 7.6: Top-1 and Top-5 by leak type, node type and network (final method).

Group	Leaks	Top-1	Top-5	Mean hops
Hidden behind another event	10	100.0%	100.0%	0.00
With sensor noise	4	100.0%	100.0%	0.00
Size variations	20	90.0%	95.0%	0.40
Start-time variations	4	100.0%	100.0%	0.00
Leak at a junction	22	100.0%	100.0%	0.00
Dead End	17	100.0%	100.0%	0.00
Hub	1	100.0%	100.0%	0.00
Inline	42	95.2%	97.6%	0.19
NET1	11	100.0%	100.0%	0.00
NET2	26	92.3%	96.2%	0.31
NET3	23	100.0%	100.0%	0.00

7.6 Effect of Leak Size

Table 7.7: Effect of hole area on the test leaks.

Area (m^2)	Leaks	Noticed by pressure	Called a leak	Top-1	Top-5
0.0001	4	75.0%	100.0%	100.0%	100.0%
0.0003	4	100.0%	100.0%	100.0%	100.0%
0.001	4	100.0%	100.0%	100.0%	100.0%
0.003	4	100.0%	100.0%	100.0%	100.0%
0.01	4	100.0%	100.0%	50.0%	75.0%

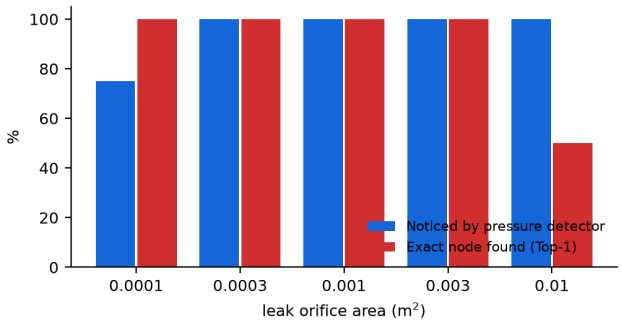


Figure 7.3: Detection and exact-node success by leak size.

Very small leaks sit close to the sensor noise, and the largest leak lowers pressure across the whole network, so its exact node is the hardest to pick. There are only four test leaks per size, so this table is indicative.

7.7 A Network Not Seen in Training

The classifier is trained on the training scenarios of two networks and tested on the test scenarios of the third (leave-one-network-out).

Table 7.8: Leave-one-network-out results.

Unseen network	Test scen.	Accuracy	Leak recall	Leak precision	Accuracy, no meters
NET1	33	48.5%	100.0%	100.0%	45.5%
NET2	44	81.8%	92.3%	100.0%	75.0%
NET3	62	54.8%	73.3%	100.0%	14.5%

Leaks carry over to a new network, but telling pump, tank, blockage and ageing events apart does not carry over well; without demand meters the accuracy on an unseen Net3 falls to 14.5%. This is the most important item for Semester 2.

7.8 Training Versus Test

The cause accuracy is 95.6% on the training scenarios and 82.7% on the test scenarios, which shows that the forest partly memorises. The leak Top-1 of the ranking is 97.8% on the 231 training leaks and 96.7% on the test leaks; the gap is small because the ranking is a fixed formula and is not trained. Table 7.9 summarises how the method evolved.

Table 7.9: Evolution of the method.

Version	Description	Result
V0	Fixed 0.5 m alarm; score = pressure drop $+ 50\times$ positive flow divergence	Top-1 58.3%, false alarms 41.7% on the test set
V1	Unaccounted flow added; pressure and flow summed in raw units	Top-1 84.9% (all scenarios): pressure drowned the flow signal
V2	Each signal scaled to 0–1, flow leads, plus neighbour smoothing	Top-1 95.9% (all scenarios)
V3 (final)	As V2 without smoothing	96.7% on the unseen test leaks

Two classifier models were compared on the same data: a depth-4 decision tree reaches 69.1% and the random forest 82.7% (with the quiet check). We did not test other model families.

7.9 Discussion and Limitations

Unaccounted flow does most of the work: if the customer use at every node is known, water that disappears with no meter behind it is a leak, and a demand change cancels out. This explains the large gain in localization over pressure alone, and why separating demand changes from leaks needs demand data. The following limitations apply.

- All data are simulated; real networks have model errors and real noise.
- The test set is small (139 scenarios, 60 scored leaks, 12 normal days) and the split was made once.
- The ranking weight and the quiet-check cut-offs were chosen before the split and not re-tuned; no hyper-parameter of the forest was tuned.
- Per-node demand meters are assumed; real utilities usually meter an area and not every node.
- Leaks inside the sensor noise are called normal; the largest leaks are hard to pinpoint.
- Pump, tank, blockage and ageing events are confused and do not transfer well to an unseen network.
- Net3 nodes 61 and 601 and the hour 21:00 are ignored; pump-speed changes were not simulated (the simulator supports only speed 1.0); only single leaks were studied.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project built LeakLens, a system that, in simulation, detects abnormal behaviour of a water network without false alarms from sensor noise, names the kind of event, and ranks the junctions where a leak most probably is. On 139 unseen test scenarios the exact leak junction was ranked first for 96.7% of the test leaks, the cause of an event was named correctly for 82.7% of the scenarios, and false alarms on normal days fell from 41.7% to 0.0%. The key idea, unaccounted flow, is simple and explainable, and the dashboard presents the result in plain language with an honest distinction between training and unseen scenarios.

8.2 Future Work

For Semester 2 we propose:

- sparse and missing sensors, using graph-based sensor placement [12, 14];
- estimating demand where nodal meters do not exist;
- several simultaneous leaks;
- cause classification that carries over to unseen networks, including graph or algorithm-informed models [13] compared against this interpretable baseline, and a comparison with gradient boosting [7];

- repeated random splits with confidence intervals, and a larger test set;
- validation on public benchmark data [4, 5] or real utility data.

References

[1] A. L. Sousa, A. F. Brochado, E. Rocha, and A. Andrade-Campos, "A state-of-the-art review of scientific, industrial, and commercial developments in water leakage management for water supply systems," *Water Research*, vol. 299, 125853, 2026.

[2] L. Romero-Ben, D. Alves, J. Blesa, G. Cembrano, V. Puig, and E. Duviella, "Leak detection and localization in water distribution networks: Review and perspective," *Annual Reviews in Control*, vol. 55, pp. 392–419, 2023.

[3] V. Vaquet, F. Hinder, A. Artelt, I. Ashraf, J. Strotherm, J. Vaquet, J. Brinkrolf, and B. Hammer, "Challenges, methods, data – a survey of machine learning in water distribution networks," in *Artificial Neural Networks and Machine Learning – ICANN 2024*, 2024.

[4] S. G. Vrachimis, M. S. Kyriakou, D. G. Eliades, and M. M. Polycarpou, "LeakDB: A benchmark dataset for leakage diagnosis in water distribution networks," in *WDSA/CCWI Joint Conference*, 2018.

[5] S. G. Vrachimis, D. G. Eliades, R. Taormina, Z. Kapelan, A. Ostfeld, S. Liu, M. Kyriakou, P. Pavlou, M. Qiu, and M. M. Polycarpou, "Battle of the leakage detection and isolation methods," *Journal of Water Resources Planning and Management*, vol. 148, no. 12, 04022068, 2022.

[6] I. Lučin, Z. Čarija, S. Družeta, and B. Lučin, "Detailed leak localization in water distribution networks using random forest classifier and pipe segmentation," *IEEE Access*, vol. 9, pp. 155113–155122, 2021.

[7] G. Marvin, L. Grbčić, S. Družeta, and L. Kranjčević, "Water distribution network leak localization with histogram-based gradient boosting," *Journal of Hydroinformatics*, 2023.

[8] L. Basnet, D. Brill, R. Ranjithan, and K. Mahinthakumar, "Supervised machine learning approaches for leak localization in water distribution systems: Impact of complexities of leak characteristics," *Journal of Water Resources Planning and Management*, vol. 149, no. 8, 2023.

[9] M. M. Rajabi, P. Komeilian, X. Wan, and R. Farmani, "Leak detection and localization in water distribution networks using conditional deep convolutional generative adversarial networks," *Water Research*, vol. 238, 120012, 2023.

[10] A. Soldevila, G. Boracchi, M. Roveri, S. Tornil-Sin, and V. Puig, "Leak detection and localization in water distribution networks by combining expert knowledge and data-driven models," *Neural Computing and Applications*, vol. 34, no. 6, pp. 4759–4779, 2022.

[11] P. Irofti, L. Romero-Ben, F. Stoican, and V. Puig, "Data-driven leak localization in water distribution networks via dictionary learning and graph-based interpolation," arXiv:2110.06372, 2021.

[12] D. U. Leonzio, T. Cattai, P. Bestagini, S. Colonnese, and S. Tubaro, "Centrality-guided node selection for efficient water leak detection," *IEEE Access*, vol. 14, pp. 24432–24446, 2026.

[13] Z. Zhang and O. Fink, "Algorithm-informed graph neural networks for leakage detection and localization in water distribution networks," *Reliability Engineering & System Safety*, vol. 265, 111494, 2026.

[14] J. J. H. van Gemert, V. Breschi, D. R. Yntema, K. J. Keesman, and M. Lazar, "The impact of sensor placement on graph-neural-network-based leakage detection," arXiv:2603.24076, 2026.

[15] O. Melnikov, Y. Dorofieiev, Y. Shakhnovskiy, H. Truong, and V. Degeler, "A multivariate statistical framework for detection, classification and pre-

localization of anomalies in water distribution networks," arXiv:2512.15685, 2025.

[16] U.S. Environmental Protection Agency, *EPANET 2.2: Water Distribution System Hydraulic and Water Quality Modeling.*

[17] U.S. Environmental Protection Agency, *WNTR: Water Network Tool for Resilience.*

Appendix A

Decision-Tree Rules

The depth-4 decision tree trained on the training scenarios (file `decision_rules.txt`). It is shown only to make the logic readable; the verdict comes from the random forest.

```
|--- unacc_conc <= 0.37
| |--- demand_frac <= 0.01
| | |--- rise_ratio <= 0.94
| | | |--- flow_frac <= 0.09
| | | | |--- class: normal
| | | |--- flow_frac > 0.09
| | | | |--- class: valve_block
| | |--- rise_ratio > 0.94
| | | |--- conc_drop <= 0.03
| | | | |--- class: tank_event
| | | |--- conc_drop > 0.03
| | | | |--- class: ageing
| |--- demand_frac > 0.01
| | |--- unacc_ratio <= 0.48
| | | |--- median_ratio <= 37.92
| | | | |--- class: demand_change
| | | |--- median_ratio > 37.92
| | | | |--- class: tank_event
| | |--- unacc_ratio > 0.48
| | | |--- conc_drop <= 0.02
| | | | |--- class: valve_block
| | | |--- conc_drop > 0.02
```

```
| | | | |--- class: normal
|--- unacc_conc > 0.37
| |--- class: leak
```

Appendix B

Example of an Analysis Response

Abridged response of `GET /api/analyze/net3_leak_15`.

```
{ "network": "net3", "split": "test", "event_class": "leak",
  "probabilities": {"leak": 0.94, ...},
  "predicted_leaks": [{"node_id": "15", "confidence_score": 1.3,
                       "pressure_drop": 6.69, "flow_divergence": 0.0199}, ...],
  "candidate_region": ["143", "15"],
  "scenario_truth": {"event_class": "leak", "leak_nodes": "15"},
  "localization_error_hops": 0 }
```